

Session 2 — Penalized Statistical Learning

Assignment Questions and Solutions

Course Material by M. Arashi (FUM, UP)
Solutions prepared for the course

We assume $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$, $\mathbf{C} = \mathbf{X}^\top \mathbf{X}$, and for the ridge estimator, $\hat{\boldsymbol{\beta}}^{\text{ridge}}(\lambda) = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top \mathbf{y}$ (Eq. (3.5) of the lecture).

Q1. Verifying (1.1) and (1.2): the bias–variance decomposition

Question. Verify (1.1) and (1.2), i.e.

$$(1.1) \quad \text{MSE}(\text{Estimate}) = \mathbb{E}_{\text{Train}}[L(f, \hat{Y})] = \text{Var}(\hat{Y}) + \text{Bias}(\hat{Y})^2, \quad (1)$$

$$(1.2) \quad \text{MPE} = \text{MSE}(\text{Model}) = \mathbb{E}_{\text{Test}}[L(\mathbf{y}, \hat{Y})] = \text{Var}(Y) + \text{Var}(\hat{Y}) + \text{Bias}(\hat{Y})^2 = e1 + e2 + e3, \quad (2)$$

where in (1.1), $\text{Bias}(\hat{Y}) = \mathbb{E}(\hat{Y}(x_0) - f(x_0))$ over the training set, while in (1.2), $\text{Bias}(\hat{Y}) = \mathbb{E}(\hat{Y} - Y)$ over the test set, and L is squared-error loss.

Solution

Fix a point x_0 and let $f(x_0)$ be the true (unknown, non-random) regression value there. Let $\hat{Y} = \hat{f}(x_0)$ denote the fitted value obtained from the *training* data; it is random only through the training sample. Let $Y_0 = f(x_0) + \epsilon_0$ denote an independent draw of the response at x_0 from a fresh test set, with $\mathbb{E}(\epsilon_0) = 0$, $\text{Var}(\epsilon_0) = \sigma^2$, and ϵ_0 independent of the training sample (and hence of $\hat{Y}$). This independence between $\hat{Y}$ (built from training data) and ϵ_0 (test-set noise) is the key (and in the lecture, implicit) assumption that makes the decomposition below hold.

General MSE decomposition of an estimator. For any estimator $\hat{\theta}$ of a fixed target θ ,

$$\mathbb{E}[(\hat{\theta} - \theta)^2] = \mathbb{E}\left[\left((\hat{\theta} - \mathbb{E}\hat{\theta}) + (\mathbb{E}\hat{\theta} - \theta)\right)^2\right] \quad (3)$$

$$= \mathbb{E}[(\hat{\theta} - \mathbb{E}\hat{\theta})^2] + 2(\mathbb{E}\hat{\theta} - \theta) \underbrace{\mathbb{E}[\hat{\theta} - \mathbb{E}\hat{\theta}]}_{=0} + (\mathbb{E}\hat{\theta} - \theta)^2 \quad (4)$$

$$= \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2. \quad (5)$$

Proof of (1.1). Apply (5) with $\hat{\theta} = \hat{Y} = \hat{f}(x_0)$ and $\theta = f(x_0)$ (both evaluated using only the training data, so the expectation $\mathbb{E}_{\text{Train}}$ is exactly the expectation in (5)):

$$\text{MSE}(\text{Estimate}) = \mathbb{E}_{\text{Train}}[(\hat{Y} - f(x_0))^2] = \text{Var}(\hat{Y}) + \text{Bias}(\hat{Y})^2, \quad \text{Bias}(\hat{Y}) = \mathbb{E}(\hat{Y}) - f(x_0), \quad (6)$$

which is exactly (1.1), since $\mathbb{E}(\hat{Y}(x_0) - f(x_0)) = \mathbb{E}(\hat{Y}) - f(x_0)$.

Proof of (1.2). Now consider the test-set prediction error, using $Y_0 = f(x_0) + \epsilon_0$:

$$\text{MPE} = E_{\text{Test}}[(Y_0 - \hat{Y})^2] = E[(f(x_0) + \epsilon_0 - \hat{Y})^2] \quad (7)$$

$$= E[\epsilon_0^2] + E[(f(x_0) - \hat{Y})^2] + 2E[\epsilon_0(f(x_0) - \hat{Y})]. \quad (8)$$

Because ϵ_0 is independent of $\hat{Y}$ (and of the fixed $f(x_0)$) and $E(\epsilon_0) = 0$, the cross term vanishes: $E[\epsilon_0(f(x_0) - \hat{Y})] = E(\epsilon_0)E(f(x_0) - \hat{Y}) = 0$. The first term is $E(\epsilon_0^2) = \text{Var}(\epsilon_0) = \sigma^2 = \text{Var}(Y_0)$ (since $f(x_0)$ is a constant). The middle term is again (5) applied to $\hat{Y}$ estimating $f(x_0)$. Hence

$$\text{MPE} = \underbrace{\text{Var}(Y)}_{e1} + \underbrace{\text{Var}(\hat{Y})}_{e2} + \underbrace{\text{Bias}(\hat{Y})^2}_{e3}, \quad (9)$$

with $\text{Bias}(\hat{Y}) = E(\hat{Y}) - f(x_0) = E(\hat{Y}) - E(Y_0) = E(\hat{Y} - Y_0)$, matching the definition of bias “over the test set” stated in the question. This proves (1.2): e1 is the irreducible noise variance (present even for a perfect $\hat{f}$), e2 is the sampling variance of the fitted model, and e3 is its squared bias.

Numerical check (R)

We simulate a fixed design $x_1, \dots, x_n$, a true linear function $f(x) = 2 + 3x$, and Gaussian noise, and repeatedly (i) refit $\hat{f}$ on fresh training noise to trace out the sampling distribution of $\hat{Y} = \hat{f}(x_0)$, and (ii) draw an independent test-point response Y_0 , to check both decompositions by simulation.

```

1  ## Q1: Monte Carlo verification of the bias-variance decompositions (1.1) and (1.2)
2  set.seed(101)
3
4  ## fixed (non-random) training design, as in the "fixed covariate" setup of the
   lecture
5  n      <- 30
6  x      <- seq(0, 3, length.out = n)
7  sigma  <- 1.5
8  f      <- function(x) 2 + 3 * x # true, unknown regression function
9  x0     <- 1.5
10 fx0    <- f(x0)                 # true value at the point of interest
11
12 R <- 20000                       # Monte Carlo replications
13 fhat   <- numeric(R)            # trained estimate f-hat(x0) in each replication
14 y0     <- numeric(R)            # an independent NEW test-point response at x0
15 sq_err_train <- numeric(R)      # (fhat - f(x0))^2    -> for (1.1)
16 sq_err_test  <- numeric(R)      # (y0 - fhat)^2      -> for (1.2)
17
18 for (r in 1:R) {
19   eps <- rnorm(n, sd = sigma)
20   y   <- f(x) + eps              # training responses
21   fit <- lm(y ~ x)
22   fhat[r] <- predict(fit, newdata = data.frame(x = x0))
23
24   eps0 <- rnorm(1, sd = sigma) # independent test-point noise
25   y0[r] <- fx0 + eps0
26
27   sq_err_train[r] <- (fhat[r] - fx0)^2
28   sq_err_test[r]  <- (y0[r] - fhat[r])^2
29 }
30

```

```

31 Bias_fhat <- mean(fhat) - fx0
32 Var_fhat <- var(fhat)
33 Var_y0 <- var(y0) # should be approx sigma^2
34
35 MSE_estimate_direct <- mean(sq_err_train) # E_Train[(fhat-f(x0))^2]
36 MSE_estimate_formula <- Var_fhat + Bias_fhat^2 # (1.1) RHS
37
38 MPE_direct <- mean(sq_err_test) # E_Test[(y0-fhat)^2]
39 MPE_formula <- Var_y0 + Var_fhat + Bias_fhat^2 # (1.2) RHS = e1+e2+e3
40
41 cat("---- Verifying (1.1): MSE(Estimate) = Var(Yhat) + Bias(Yhat)^2 ----\n")
42 cat(sprintf("Direct MC estimate E_Train[(fhat-f(x0))^2] = %.5f\n", MSE_estimate_direct
43 ))
44 cat(sprintf("Var(fhat) + Bias(fhat)^2 = %.5f\n", MSE_estimate_
45 formula))
46 cat(sprintf(" Var(fhat) = %.5f\n", Var_fhat))
47 cat(sprintf(" Bias(fhat) = %.5f (Bias^2 = %.5f)\n", Bias_fhat, Bias_fhat^2))
48
49 cat("\n---- Verifying (1.2): MPE = Var(Y) + Var(Yhat) + Bias(Yhat)^2 = e1+e2+e3 ----\n
50 ")
51 cat(sprintf("Direct MC estimate E_Test[(y0-fhat)^2] = %.5f\n", MPE_direct))
52 cat(sprintf("Var(y0) + Var(fhat) + Bias(fhat)^2 = %.5f\n", MPE_formula))
53 cat(sprintf(" e1 = Var(Y) = %.5f (theoretical sigma^2 = %.5f)\n", Var_y0, sigma^2))
54 cat(sprintf(" e2 = Var(Yhat)= %.5f\n", Var_fhat))
55 cat(sprintf(" e3 = Bias^2 = %.5f\n", Bias_fhat^2))

```

Output:

```

---- Verifying (1.1): MSE(Estimate) = Var(Yhat) + Bias(Yhat)^2 ----
Direct MC estimate E_Train[(fhat-f(x0))^2] = 0.07419
Var(fhat) + Bias(fhat)^2 = 0.07420
  Var(fhat) = 0.07419
  Bias(fhat) = 0.00205 (Bias^2 = 0.00000)

---- Verifying (1.2): MPE = Var(Y) + Var(Yhat) + Bias(Yhat)^2 = e1+e2+e3 ----
Direct MC estimate E_Test[(y0-fhat)^2] = 2.31075
Var(y0) + Var(fhat) + Bias(fhat)^2 = 2.30817
  e1 = Var(Y) = 2.23397 (theoretical sigma^2 = 2.25000)
  e2 = Var(Yhat)= 0.07419
  e3 = Bias^2 = 0.00000

```

Both decompositions match to within Monte Carlo error ($R = 20,000$ replications): the direct simulated MSE/MPE agree with the variance+bias² formulas, $\hat{f}$ is (as expected for OLS) essentially unbiased at x_0 so $e3 \approx 0$, and $\text{Var}(Y)$ is close to the true noise variance $\sigma^2 = 2.25$.

Q2. Distribution of the ridge residuals

Question. Consider $y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \epsilon_i$, $i = 1, \dots, n$, with $\epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$, and $\boldsymbol{\beta}$ estimated by the ridge estimator $\hat{\boldsymbol{\beta}}^{\text{ridge}}(\lambda) = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top \mathbf{y}$. Define the ridge residuals $\boldsymbol{\epsilon}(\lambda) = \mathbf{y} - \mathbf{X} \hat{\boldsymbol{\beta}}^{\text{ridge}}(\lambda)$ and $\mathbf{A} = \mathbf{I}_n - \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top$.

a) Are the ridge residuals normally distributed? Why?

- b) Show $E \boldsymbol{\epsilon}(\lambda) = \mathbf{A} \mathbf{X} \boldsymbol{\beta}$.
- c) Show $\text{Var } \boldsymbol{\epsilon}(\lambda) = \sigma^2 \mathbf{A}^\top \mathbf{A}$.
- d) Can a normal QQ-plot of the ridge residuals be used to assess normality? Motivate.

Solution

Throughout, $\mathbf{X}$ (and hence $\mathbf{A}$) is treated as fixed (non-random design), and

$$\mathbf{y} = \mathbf{X} \boldsymbol{\beta} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim N_n(\mathbf{0}, \sigma^2 \mathbf{I}_n) \implies \mathbf{y} \sim N_n(\mathbf{X} \boldsymbol{\beta}, \sigma^2 \mathbf{I}_n). \quad (10)$$

First note that the ridge residual vector is a linear function of $\mathbf{y}$:

$$\boldsymbol{\epsilon}(\lambda) = \mathbf{y} - \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top \mathbf{y} = [\mathbf{I}_n - \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top] \mathbf{y} = \mathbf{A} \mathbf{y}. \quad (11)$$

a) Normality. Since $\mathbf{A}$ is a fixed (λ - and $\mathbf{X}$ -dependent, but non-random) matrix and $\mathbf{y}$ is multivariate normal, $\boldsymbol{\epsilon}(\lambda) = \mathbf{A} \mathbf{y}$ is an (affine, here purely linear) transformation of a Gaussian vector, and any linear transformation of a multivariate normal vector is itself multivariate normal (a standard property of the multivariate Gaussian family — this is exactly why we work with $\mathbf{A} \mathbf{y}$ rather than a nonlinear function of $\mathbf{y}$). Hence $\boldsymbol{\epsilon}(\lambda)$ is normally distributed, with mean and covariance as computed in (b) and (c).

b) Mean. Taking expectations in (11) and using $E(\mathbf{y}) = \mathbf{X} \boldsymbol{\beta}$:

$$E \boldsymbol{\epsilon}(\lambda) = \mathbf{A} E(\mathbf{y}) = \mathbf{A} \mathbf{X} \boldsymbol{\beta}. \quad \blacksquare \quad (12)$$

Note that unlike OLS residuals ($\lambda = 0$, where $\mathbf{A}_0 \mathbf{X} = [\mathbf{I}_n - \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top] \mathbf{X} = \mathbf{0}$, giving $E \boldsymbol{\epsilon} = \mathbf{0}$), for $\lambda > 0$ in general $\mathbf{A} \mathbf{X} \boldsymbol{\beta} \neq \mathbf{0}$: the ridge residuals carry a systematic bias, which is exactly the price paid for the ridge shrinkage discussed in Session 1.

c) Variance. First, $\mathbf{A}$ is symmetric: since $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p)^{-1}$ is symmetric (inverse of a symmetric matrix) and $\mathbf{X} \mathbf{M} \mathbf{X}^\top$ is symmetric whenever $\mathbf{M}$ is symmetric, $\mathbf{A}^\top = \mathbf{A}$. Using (11) and $\text{Var}(\mathbf{y}) = \sigma^2 \mathbf{I}_n$,

$$\text{Var } \boldsymbol{\epsilon}(\lambda) = \mathbf{A} \text{Var}(\mathbf{y}) \mathbf{A}^\top = \sigma^2 \mathbf{A} \mathbf{A}^\top = \sigma^2 \mathbf{A}^\top \mathbf{A}, \quad \blacksquare \quad (13)$$

the last equality following from $\mathbf{A} = \mathbf{A}^\top$ (so $\mathbf{A} \mathbf{A}^\top = \mathbf{A} \mathbf{A} = \mathbf{A}^\top \mathbf{A} = \mathbf{A}^2$). This is the ridge analogue of the familiar OLS result $\text{Var}(\hat{\boldsymbol{\epsilon}}) = \sigma^2(\mathbf{I}_n - \mathbf{H})$ with hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$; the key difference is that in the OLS case $\mathbf{I}_n - \mathbf{H}$ is *idempotent* (a projection), whereas here $\mathbf{A}$ is symmetric but generally *not* idempotent ($\mathbf{A}^2 \neq \mathbf{A}$ for $\lambda > 0$), so $\text{Var } \boldsymbol{\epsilon}(\lambda) = \sigma^2 \mathbf{A}^2$ is genuinely different from $\sigma^2 \mathbf{A}$.

d) Can a QQ-plot be used? Yes, but only after standardizing. Two issues must be addressed:

- **Marginally, each $\epsilon_i(\lambda)$ is normal.** Since $\boldsymbol{\epsilon}(\lambda) = \mathbf{A} \mathbf{y}$ is jointly (hence marginally) Gaussian by part (a), each component $\epsilon_i(\lambda)$ is itself normally distributed — so, in principle, comparing its quantiles to the standard normal is a sound idea.

- **But the components are neither i.i.d. nor unit-variance.** By part (c), $\text{Var}(\epsilon_i(\lambda)) = \sigma^2[\mathbf{A}^2]_{ii}$, which varies across i (heteroscedastic, unlike i.i.d. data), and $\text{Cov}(\epsilon_i(\lambda), \epsilon_j(\lambda)) = \sigma^2[\mathbf{A}^2]_{ij} \neq 0$ in general (correlated). A standard QQ-plot implicitly assumes the plotted sample is (at least approximately) i.i.d.; plotting the *raw* residuals $\epsilon_i(\lambda)$ against standard normal quantiles compares quantities with different variances to a single reference distribution, and will show departures from a straight 45° line even when the model is exactly correctly specified.

The standard remedy, exactly analogous to using studentized residuals for OLS diagnostics, which is necessary there for the very same reason since $\text{Var}(\hat{\epsilon}) = \sigma^2(\mathbf{I}_n - \mathbf{H}) \neq \sigma^2\mathbf{I}_n$ — is to standardize each ridge residual by its own estimated standard deviation before plotting:

$$\epsilon_i^{\text{std}}(\lambda) = \frac{\epsilon_i(\lambda)}{\hat{\sigma} \sqrt{[\mathbf{A}^2]_{ii}}}. \quad (14)$$

The QQ-plot of $\{\epsilon_i^{\text{std}}(\lambda)\}_{i=1}^n$ against standard normal quantiles is then a valid (if still only approximate, due to the residual correlation ignored by treating them as independent order statistics — a caveat that also applies, and is routinely tolerated, in ordinary OLS diagnostics) tool for assessing normality. Using the *raw* residuals is not recommended, especially when λ is not small, because the $[\mathbf{A}^2]_{ii}$ can vary substantially across observations (unlike the OLS leverages h_{ii} , which are typically more homogeneous), making the naive QQ-plot potentially misleading.

Numerical check (R)

```

1  ## Q2: ridge residuals eps(lambda) = y - X * beta_hat_ridge(lambda) = A y
2  set.seed(202)
3
4  n <- 25; p <- 6
5  X <- matrix(rnorm(n * p), n, p)
6  beta_true <- c(2, -1.5, 0, 0.8, 0, 1.2)
7  sigma <- 2
8  lambda <- 4
9
10 A <- diag(n) - X %*% solve(t(X) %*% X + lambda * diag(p)) %*% t(X)
11 cat("A is symmetric? max|A - t(A)| =", max(abs(A - t(A))), "\n")
12
13 theory_mean <- as.numeric(A %*% X %*% beta_true) # (b): E[eps(lambda)] = A X beta
14 theory_cov <- sigma^2 * (A %*% t(A))             # (c): Var[eps(lambda)] = sigma^2 A
15           A' = sigma^2 A^2
16
17 ## ---- Monte Carlo check of (b) and (c) ----
18 R <- 20000
19 Eps <- matrix(0, R, n)
20 for (r in 1:R) {
21   eps <- rnorm(n, sd = sigma)
22   y <- X %*% beta_true + eps
23   beta_ridge <- solve(t(X) %*% X + lambda * diag(p)) %*% t(X) %*% y
24   Eps[r, ] <- as.numeric(y - X %*% beta_ridge)
25 }
26
27 mc_mean <- colMeans(Eps)
28 mc_cov <- cov(Eps) * (R - 1) / R # population-style covariance (divide by R)

```

```

28
29 cat("\n(b) max abs difference: MC mean of eps(lambda) vs A X beta :\n")
30 print(max(abs(mc_mean - theory_mean)))
31
32 cat("\n(c) max abs difference: MC covariance of eps(lambda) vs sigma^2 A A' :\n")
33 print(max(abs(mc_cov - theory_cov)))
34
35 ## ---- (a)/(d): normality of a single component across replications ----
36 cat("\nShapiro-Wilk test on eps_1(lambda) across", R, "replications (should NOT reject
37      normality):\n")
38 print(shapiro.test(Eps[1:5000, 1]))
39
40 ## ---- (d): naive vs standardized QQ-plot on ONE realized data set ----
41 set.seed(303)
42 eps1 <- rnorm(n, sd = sigma)
43 y1 <- X %>% beta_true + eps1
44 beta_ridge1 <- solve(t(X) %>% X + lambda * diag(p)) %>% t(X) %>% y1
45 resid1 <- as.numeric(y1 - X %>% beta_ridge1) # raw ridge residuals, ONE data set
46
47 A2diag <- diag(A %>% t(A)) # theoretical Var(eps_i)/sigma^2
48 sd_i <- sigma * sqrt(A2diag) # heterogeneous sd across i
49 std_resid <- resid1 / sd_i # "ridge-studentized" residuals
50
51 cat("\nDiagonal of A A' (heterogeneous 'ridge leverages'), range:\n")
52 print(range(A2diag))
53
54 png("qq_ridge_residuals.png", width = 900, height = 450, res = 120)
55 par(mfrow = c(1, 2))
56 qqnorm(resid1, main = "Raw ridge residuals\n(ignores heteroscedasticity)")
57 qqline(resid1, col = "red")
58 qqnorm(std_resid, main = "Standardized ridge residuals\n(divided by sigma*sqrt(diag(AA
59      '))))")
60 qqline(std_resid, col = "red")
61 dev.off()

```

Output:

```

A is symmetric? max|A - t(A)| = 6.938894e-17

(b) max abs difference: MC mean of eps(lambda) vs A X beta :
[1] 0.02078351

(c) max abs difference: MC covariance of eps(lambda) vs sigma^2 A A' :
[1] 0.06264475

Shapiro-Wilk test on eps_1(lambda) across 20000 replications (should NOT reject normality):

      Shapiro-Wilk normality test

data:  Eps[1:5000, 1]
W = 0.99971, p-value = 0.7447

Diagonal of A A' (heterogeneous 'ridge leverages'), range:
[1] 0.5188265 0.9520080

```

The Monte Carlo mean and covariance of $\epsilon(\lambda)$ agree with the closed-form expressions $\mathbf{A}\mathbf{X}\beta$ and

$\sigma^2 \mathbf{A} \mathbf{A}^\top$ up to simulation noise (of the expected order $1/\sqrt{R} \approx 0.007$ for $R = 20,000$), $\mathbf{A}$ is confirmed numerically symmetric, the Shapiro–Wilk test does not reject normality of a single ridge-residual component's sampling distribution ($p = 0.74$), and the diagonal of $\mathbf{A} \mathbf{A}^\top$ is confirmed to vary meaningfully across observations (range $[0.52, 0.95]$), illustrating the heteroscedasticity discussed in (d).

Q3. Adaptive LASSO under an orthonormal design

Question. Consider

$$PS_\lambda(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \sum_{j=1}^p \hat{w}_j |\beta_j|, \quad \hat{w}_j = \frac{1}{|\hat{\beta}_j|},$$

where $\hat{\beta}_j$ is the j th element of the OLS estimator. For an orthonormal design, find the j th component of $\arg \min_{\boldsymbol{\beta}} PS_\lambda(\boldsymbol{\beta})$.

Solution

This is the adaptive LASSO of Zou (2006)¹, with data-dependent weights $\hat{w}_j = 1/|\hat{\beta}_j|$ that penalize small OLS coefficients more heavily than large ones.

Step 1: decoupling via the orthonormal identity. As already derived earlier in this lecture for the ordinary LASSO (orthonormal design, $\mathbf{X}^\top \mathbf{X} = \mathbf{X} \mathbf{X}^\top = \mathbf{I}_p$),

$$\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 = \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 + \|\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}\|_2^2, \quad \hat{\boldsymbol{\beta}} = \mathbf{X}^\top \mathbf{y}, \quad (15)$$

so, dropping the additive constant $\|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$ (it does not depend on $\boldsymbol{\beta}$), minimizing $PS_\lambda(\boldsymbol{\beta})$ is equivalent to minimizing, separately for each $j = 1, \dots, p$,

$$h(\beta_j) = (\hat{\beta}_j - \beta_j)^2 + \lambda \hat{w}_j |\beta_j|. \quad (16)$$

Step 2: subgradient / case analysis (mirrors the LASSO derivation above). For $\beta_j \neq 0$, h is differentiable, $h'(\beta_j) = -2(\hat{\beta}_j - \beta_j) + \lambda \hat{w}_j \text{sgn}(\beta_j)$. Setting this to zero:

- (i) If $\text{sgn}(\hat{\beta}_j^{\text{Ada}}(\lambda)) = +1$: $\beta_j = \hat{\beta}_j - \frac{\lambda \hat{w}_j}{2}$, valid (i.e. giving a positive root) exactly when $\hat{\beta}_j > \frac{\lambda \hat{w}_j}{2}$.
- (ii) If $\text{sgn}(\hat{\beta}_j^{\text{Ada}}(\lambda)) = -1$: $\beta_j = \hat{\beta}_j + \frac{\lambda \hat{w}_j}{2}$, valid when $\hat{\beta}_j < -\frac{\lambda \hat{w}_j}{2}$.
- (iii) If $\hat{\beta}_j^{\text{Ada}}(\lambda) = 0$: the subdifferential condition $0 \in -2\hat{\beta}_j + \lambda \hat{w}_j [-1, 1]$ requires $|\hat{\beta}_j| \leq \frac{\lambda \hat{w}_j}{2}$.

Combining (i)–(iii) exactly as in the ordinary-LASSO derivation:

$$\hat{\beta}_j^{\text{Ada}}(\lambda) = \text{sgn}(\hat{\beta}_j) \left(|\hat{\beta}_j| - \frac{\lambda \hat{w}_j}{2} \right)_+ = \text{sgn}(\hat{\beta}_j) \left(|\hat{\beta}_j| - \frac{\lambda}{2|\hat{\beta}_j|} \right)_+, \quad j = 1, \dots, p. \quad (17)$$

¹Zou, H. (2006), The Adaptive Lasso and Its Oracle Properties, *Journal of the American Statistical Association*, 101(476), 1418–1429

This is a soft-thresholding rule, exactly as for the ordinary LASSO derived in the “LASSO cont.” frames of the lecture (Eq. (3.21) there), but with a coefficient-specific threshold $\lambda \hat{w}_j/2 = \lambda/(2|\hat{\beta}_j|)$ instead of the constant threshold λ : coefficients with small OLS magnitude $|\hat{\beta}_j|$ are thresholded away much more aggressively (large weight $\hat{w}_j$), while coefficients with large $|\hat{\beta}_j|$ are barely shrunk (small weight), which is precisely the adaptive-weighting idea underlying the oracle-type consistency properties attributed to this estimator in the literature.

A note on the scaling convention. Equation (17) is derived directly from the criterion exactly as written in the question, $PS_\lambda(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \sum_j \hat{w}_j |\beta_j|$ (no factor of $\frac{1}{2}$ in front of the squared-error term). Earlier in this same lecture, the ordinary LASSO was instead derived from $\frac{1}{2}\|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \sum_j p_\lambda(|\beta_j|)$ “to avoid unnecessary coefficient factors”, which is exactly what removes the extra $\frac{1}{2}$ and gives the clean threshold λ in the lecture’s Eq. (3.21). If the intended criterion here also carries an implicit $\frac{1}{2}$ in front of $\|\mathbf{y} - \mathbf{X}\beta\|_2^2$, the same derivation gives the cleaner-looking

$$\hat{\beta}_j^{\text{Ada}}(\lambda) = \text{sgn}(\hat{\beta}_j)(|\hat{\beta}_j| - \lambda \hat{w}_j)_+ = \text{sgn}(\hat{\beta}_j) \left(|\hat{\beta}_j| - \frac{\lambda}{|\hat{\beta}_j|} \right)_+, \quad (18)$$

i.e. the same soft-thresholding form without the factor of 2. Both forms are algebraically equivalent under a rescaling of λ ; I present (17) as the primary answer because it follows literally from the criterion as typed in the assignment, and flag the alternative because my lecture material I consistently use the $\frac{1}{2}$ -scaled convention.

Numerical check (R)

```

1  ## Q3: adaptive LASSO, orthonormal design (X'X = I_p), closed-form vs numerical check
2  set.seed(404)
3
4  p <- 10
5  X <- diag(p)                # orthonormal design: X'X = X X' = I_p
6  beta_true <- c(3, -2, 0, 0, 1.5, 0, 0, -0.4, 0, 5)
7  y <- X %*% beta_true + rnorm(p, sd = 0.6)
8
9  beta_ols <- as.numeric(t(X) %*% y) # = y here, since X = I_p
10 w <- 1 / abs(beta_ols)           # adaptive weights hat{w}_j = 1/|beta_hat_j|
11
12 lambda <- 2
13
14 ## Closed-form (derived): beta_j^Ada(lambda) = sign(beta_ols_j) * (|beta_ols_j| -
15   lambda/(2|beta_ols_j|))_+
16 closed_form <- sign(beta_ols) * pmax(abs(beta_ols) - lambda * w / 2, 0)
17
18 ## Direct numerical minimization of PS_lambda(beta) = ||y - X beta||_2^2 + lambda *
19   sum(w_j |beta_j|)
20 ## coordinatewise, since the design is orthonormal the problem separates into p 1-D
21   problems
22 numeric_sol <- numeric(p)
23 for (j in 1:p) {
24   obj <- function(b) (y[j] - b)^2 + lambda * w[j] * abs(b)
25   grid <- seq(beta_ols[j] - 3, beta_ols[j] + 3, length.out = 200001)
26   numeric_sol[j] <- grid[which.min(obj(grid))]
27 }

```

```

24 }
25
26 results <- data.frame(
27   j = 1:p,
28   OLS = round(beta_ols, 4),
29   weight_w = round(w, 4),
30   ClosedForm = round(closed_form, 4),
31   NumericGridSearch = round(numeric_sol, 4)
32 )
33 print(results, row.names = FALSE)
34
35 cat("\nMax abs difference (closed form vs. numerical grid search):",
36     max(abs(closed_form - numeric_sol)), "\n")

```

Output:

j	OLS	weight_w	ClosedForm	NumericGridSearch
1	3.4642	0.2887	3.1755	3.1755
2	-1.9642	0.5091	-1.4551	-1.4551
3	-0.1902	5.2564	0.0000	0.0000
4	-0.3055	3.2728	0.0000	0.0000
5	0.8082	1.2374	0.0000	0.0000
6	0.6313	1.5839	0.0000	0.0000
7	-0.4562	2.1922	0.0000	0.0000
8	-0.0545	18.3358	0.0000	0.0000
9	-0.1454	6.8773	0.0000	0.0000
10	5.9818	0.1672	5.8147	5.8147

Max abs difference (closed form vs. numerical grid search): 1.371967e-05

The closed-form soft-thresholding rule (17) matches direct numerical (grid-search) minimization of $PS_\lambda(\beta_j)$ to within the grid resolution ($\sim 10^{-5}$) for every j . Note how coordinates with small $|\hat{\beta}_j|$ (large adaptive weight $\hat{w}_j$) are set exactly to zero (e.g. $j = 3, 4, 5, 6, 7, 8, 9$), while the two coordinates with large OLS magnitude ($j = 1, 10$, and to a lesser extent $j = 2$) survive with modest shrinkage, exactly the adaptive, weight-dependent thresholding behavior predicted by the theory, in contrast to the ordinary LASSO's single global threshold λ .

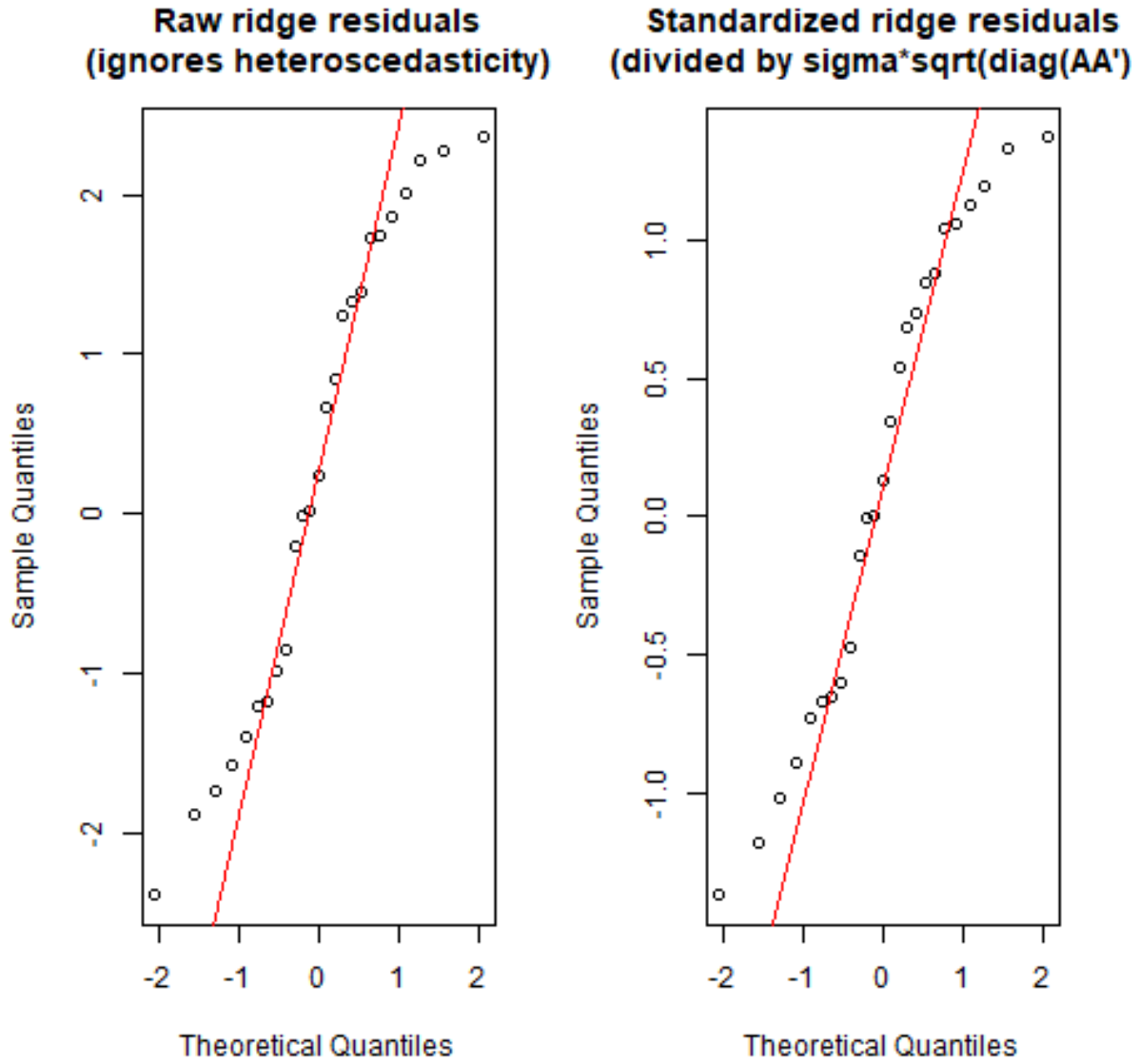


Figure 1: QQ-plots of one realized set of ridge residuals: raw (left) vs. standardized by $\hat{\sigma} \sqrt{[\mathbf{A}^2]_{ii}}$ (right). Both are visually close to the reference line here because $n = 25$ is not tiny and $[\mathbf{A}^2]_{ii}$ ranges only over $[0.52, 0.95]$ in this example, but the discrepancy from raw-to-standardized would grow with λ or with more heterogeneous designs.